

денного множества процессов тех работ, которые необходимы для реализации конкретного программного проекта. Однако практические рекомендации по организации внедрения ГОСТ 12207—2010 остаются за границами самого документа.

2.3. Модели жизненного цикла программного обеспечения

2.3.1. Каскадная модель

Повторимся, что каскадная модель жизненного цикла, также называемая моделью «водопада» (англ. *waterfall*), была разработана еще в 1980-х гг. и на протяжении многих лет считалась стандартом для разработки ПО. Данная модель характеризуется тем, что этапы строго последовательны и переход между ними необратим (рис. 2.6). Это означает, что в рамках каскадной модели переход к следующему этапу (например, от проектирования и сбора требований к разработке и развертыванию) может произойти только по завершении предыдущего этапа. Модель «водопада» была применена одной из первых, и одно из ее основных достоинств в возможности планирования сроков и стоимости каждого этапа. Однако на практике разработка системы почти никогда не проходит строго в соответствии с жесткой, заранее продуманной схемой. В частности, это касается сбора требований, так как реально при старте проекта требования бывают определены только частично и в дальнейшем уточняются, изменяются и дополняются. К тому же, если изначально требования были определены неточно, высока вероятность того, что система не будет удовлетворять потребностям заказчика.

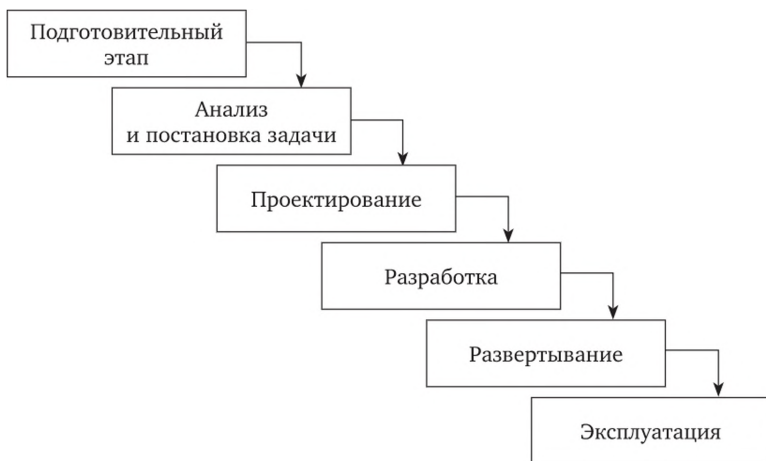


Рис. 2.6. Каскадная модель разработки ПО

Далее сравним преимущества и недостатки каскадных моделей разработки ПО.

Преимущества	Недостатки
Последовательное выполнение этапов проекта в фиксированном порядке. Наличие полной и согласованной документации на каждом этапе	Задержка при получении значимого для заказчика результата. Слишком высокая цена ошибки (даже в модели с промежуточным контролем)

2.3.2. Каскадная модель с промежуточным контролем

Чтобы предусмотреть возможность возвращения к предыдущим этапам для внесения определенных изменений и пересмотра отдельных вопросов, в качестве одной из вариаций каскадной модели была создана *каскадная модель с промежуточным контролем*. Она предполагает увеличенное время, отведенное на разработку, за счет проведения промежуточных корректировок между фазами жизненного цикла (рис. 2.7). Это снижает риски получения некачественного продукта на выходе и повышает надежность системы в целом.

Важно отметить, что согласование результатов в двух описанных моделях происходит только по окончании внедрения, а значит, повышается вероятность получения программно продукта, который морально устареет либо не будет востребован рынком. Еще больше увеличивают риски возможные неточности в исходном техническом задании. В итоге можно говорить о проблеме задержки в получении результата, которая не может быть решена в «каскадном» варианте разработки и внедрения системы.

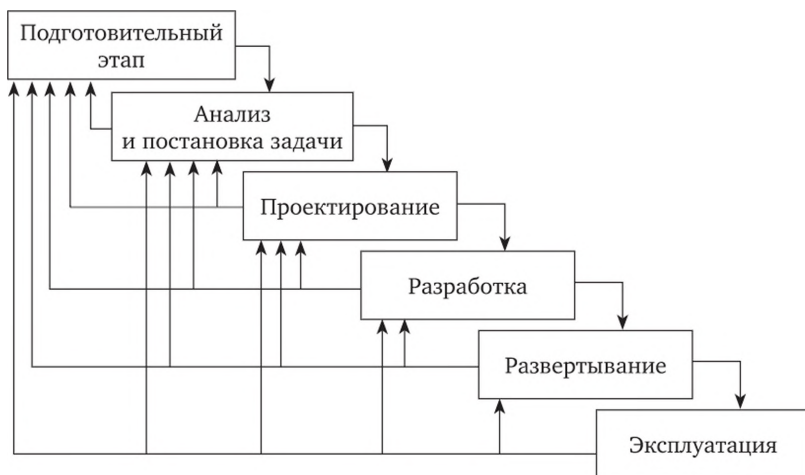


Рис. 2.7. Каскадная модель разработки ПО с промежуточным контролем

2.3.3. Метод разработки через тестирование (V-модель)

Для установления связей между этапами каскадной модели иногда используют метод разработки, получивший название V-модель. Приближенная по своей сути к практикам PRINCE2 (широко применяющаяся методология управления программами и проектами в организации), V-модель разработки через тестирование была разработана еще в конце 1980-х гг. ведомствами Германии и США и до сих пор является стандартом немецких правительственных и оборонных проектов. Основной ее принцип состоит в постепенном возрастании степени детализации проекта с течением времени и одновременном проведении «горизонтальных» итераций. Таким образом (рис. 2.8), результаты каждой из фаз левой стороны буквы V влияют на тестирование и компоновку проекта с правой стороны буквы V: приемо-сдаточные испытания основываются на проведенном анализе требований, интеграционное тестирование — на высокоуровневом описании архитектуры, модульное тестирование — на архитектуре, интерфейсах, алгоритмах и прочих элементах детализированных требований к системе.

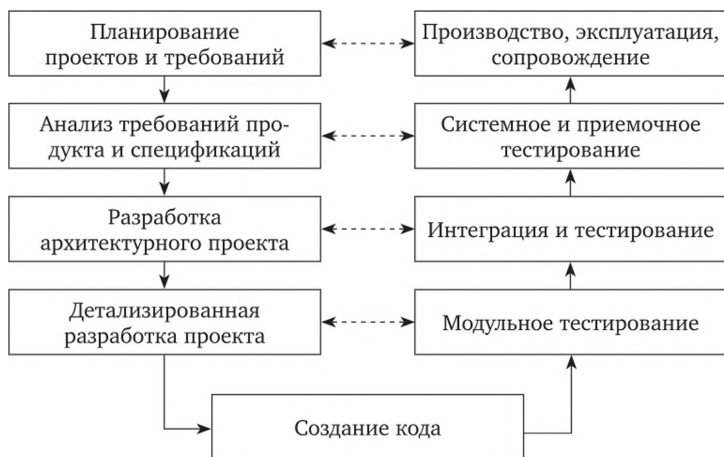


Рис. 2.8. V-модель разработки ПО

Важна гибкость данной модели, так как по сути она адаптируема под любой тип организации. Промежуточные результаты проверяются на ранних стадиях, и минимизация рисков достигается благодаря простому соотношению фаз. V-модель не рассматривает непосредственно стадию обслуживания и утилизации, учитывая лишь активности по подготовке к ним. Сам по себе этот метод часто в литературе не рассматривается как отдельная модель жизненного цикла, но удобен в установлении соответствия между фазами.

Далее сравним преимущества и недостатки V-модели разработки ПО:

Преимущества	Недостатки
Модель предусматривает верификацию и валидацию всех внешних и внутренних данных. Подходит для проектов, в которых надежность и безопасность важнее стоимости	Модель не поддерживает внесение динамических изменений при разработке. Модель не предназначена для управления параллельными событиями

2.3.4. Спиральная модель

Для нивелирования рисков, связанных с вышеописанной проблемой, была создана **спиральная модель** (еще называемая **итерационной**). Фазы жизненного цикла не последовательны, т. е. допустимо (но не обязательно!) начало работ над следующим этапом до завершения предыдущего (рис. 2.9). Таким образом, суть спиральной модели состоит в возможности прохождения всех этапов жизненного цикла системы в несколько итераций. При этом на каждой итерации создается новый прототип и проверяется актуальность требований, по которым он создавался, вносятся технические доработки в интерфейс и функциональность. Подобная гибкость позволяет использовать модель на предприятиях любого масштаба.

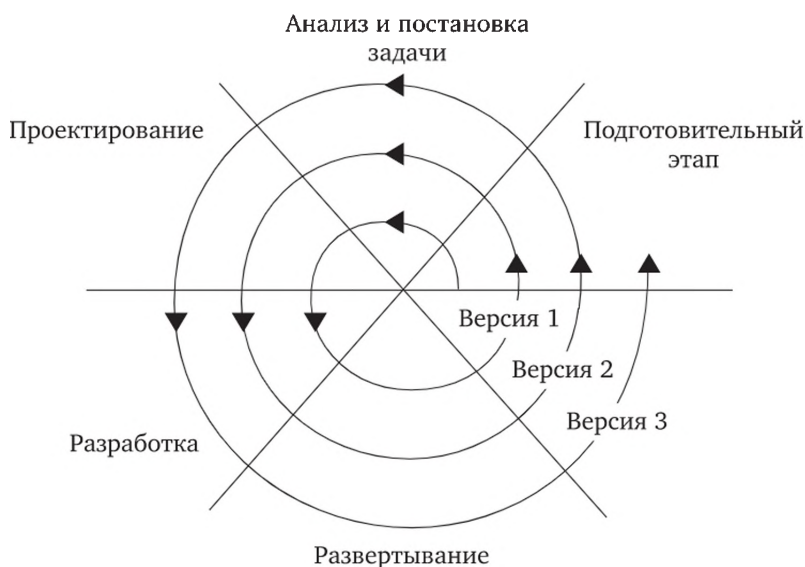


Рис. 2.9. Спиральная модель разработки ПО

Спиральную модель возможно применять в областях, где требуется завершенность разработки в выпускаемых версиях. В част-

ности, это разработки ПО в области оборонной промышленности, авиастроения, атомной энергетики и другие отрасли, где любая ошибка может стоить человеческих жизней. На практике предполагается, что применение спиральной модели предполагает создание единственной работоспособной версии только в конце всего цикла.

В российской практике часто название спиральной модели присваивается также итерационной и эволюционным моделям, хотя западная практика приводит различия в определениях и применении этих моделей. В отличие от спиральной модели, итерационная и эволюционная модель предполагают наличие промежуточных версий, которые может использовать конечный пользователь. Эти модели будут рассмотрены ниже.

Сравним преимущества и недостатки спиральной модели разработки:

Преимущества	Недостатки
Высокое внимание к рискам и анализ рисков позволяют избежать дополнительных затрат. Все фазы разработки взаимосвязаны, предусмотрены итерации	Не может эффективно применяться в сложных проектах с ожиданием работоспособных промежуточных версий. Сложная структура модели для реализации

2.3.5. Итерационная модель

Итерационная и инкрементная модели, которые будут рассмотрены далее, на первый взгляд кажутся похожими. Однако существует ряд принципов, которые отличают их друг от друга.

Итерационная модель предполагает, что в начале проекта реализуется часть функциональности, которая становится базой для определения дальнейших требований. Затем процесс повторяется, на каждой фазе к продукту добавляется новая функциональность, а вместе с этим определяются и дальнейшие требования. Не требуется, чтобы каждая версия была идеальной, однако по мере разработки ее соответствие потребностям заказчика должно неуклонно повышаться.

Инкрементная модель ведет команду разработчиков другим путем. Продукт разбивается на набор взаимосвязанных модулей, каждый из которых разрабатывается отдельно. По мере разработки каждого нового модуля он включается в единый продукт.

На рис. 2.10 показаны итерационный и инкрементный процесс «создания» Моны Лизы. В случае использования итерационной модели сначала делается карандашный набросок картины. Затем, на второй итерации, к наброску добавляются базовые цвета. И, наконец, на третьей итерации добавляются детали, картина становится насыщенной и принимает заверченный вид.

Итерационная модель



Инкрементная модель



Рис. 2.10. Различие между итерационной и инкрементной моделями

А в случае использования инкрементной модели картина условно разбивается на несколько частей. Каждая часть — это целостный элемент, который в дальнейшем объединяется с другими элементами для получения конечного результата. Каждая из этих моделей предполагает создание новых версий продукта, которые могут передаваться в эксплуатацию.

Итерационная модель основана на базовой идее разработки ПО через повторяющиеся фазы, каждая из которых длится относительно небольшой промежуток времени. Это позволяет использовать опыт, полученный в ходе выполнения ранних фаз, для более продуктивного выполнения последующих фаз. Результаты выполнения каждой фазы тщательно анализируются для корректировки последующих фаз.

Каждая фаза содержит набор повторяющихся активностей, причем в зависимости от фазы тому или иному виду активностей уделяется разное внимание. К примеру, на ранних фазах большое внимание уделяется сбору требований, тогда как на поздних — реализации, тестированию и развертыванию.

Как правило, итерационная модель применяется при разработке нетиповых систем. Она позволяет быстро подготовить прототип. Часто наличие прототипа крайне помогает в ситуациях, когда требуется разъяснить и уточнить требования, выбрать концептуальное решение или даже в целом определить целесообразность реализации проекта. При этом сам прототип может моделировать либо исключительно пользовательский интерфейс, либо архитектурную сторону системы (логику обработки и хранения данных).

В первом случае важно принимать во внимание возможное сопротивление со стороны пользователей, которые хотят видеть требующуюся именно им функциональность в первом же прототипе, а затем — в первой же предоставленной им версии, а не второй или третьей. Соответственно, на первый план выходит необходимость грамотного управления ожиданиями пользователей.

Это означает, что процесс создания системы и само управление проектом становится более гибким и управляемым, с совершенствованием системы на каждой новой фазе. Уменьшаются риски (в том числе финансовые) для заказчика системы, которые могут отказаться от проекта еще на этапе показа первого прототипа в случае абсолютного его несоответствия ожиданиям и потребностям (либо в случае изменения рыночной ситуации).

Как пример, итерационная модель разработки ПО применяется в корпоративной методологии Rational Unified Process (RUP). На рис. 2.11 отражена итерационная модель с фазами и дисциплинами RUP.

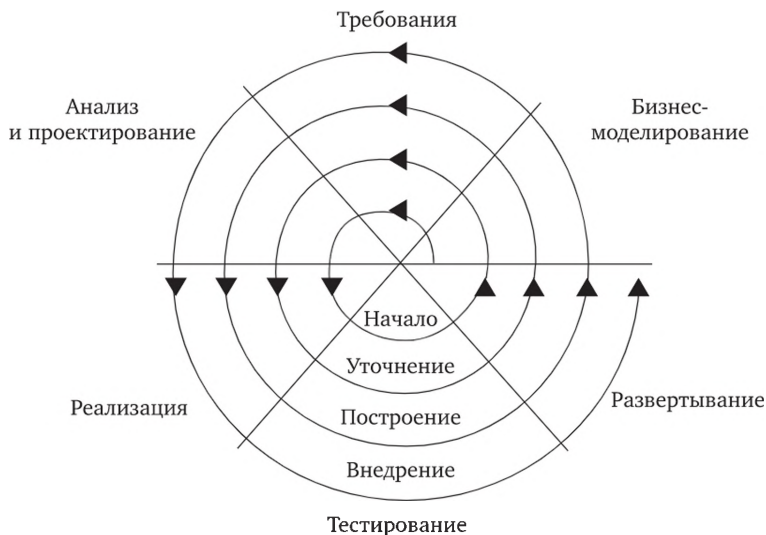


Рис. 2.11. Итерационная модель разработки ПО на примере RUP

Важно отметить, что набирающие сегодня популярность модели гибкой разработки построены на основе итерационной модели. Особенностью этих моделей является то, что команда разработчиков находится в условиях, когда они должны как можно чаще осуществлять сборку версии продукта, которая может быть передана в эксплуатацию. В некоторых методологиях, таких как Scrum, команда разработки должны формировать версию работающего продукта через определенные промежутки времени (например, каждые две недели).

Сравним преимущества и недостатки итерационной модели разработки ПО:

Преимущества	Недостатки
Эффективная обратная связь команды разработчиков с заказчиками. Раннее обнаружение конфликтов между требованиями, моделями и реализацией продукта. Затраты равномерно распределены по всему проекту, а нагрузка — по всем участникам	Требуется сильная вовлеченность заказчика в процесс разработки ПО. Нестабильный список требований. Высокая сложность планирования и повышенная нагрузка на менеджера проекта

2.3.6. Инкрементная модель

Инкрементная модель разработки ПО — это метод разработки программного обеспечения, в котором продукт разрабатывается, релизируется и тестируется эволюционно (с небольшими добавлениями за промежутки времени). Продукт считается завершенным в случае, когда он соответствует всем предъявляемым к нему требованиям.

Продукт декомпозируется на набор модулей, каждый из которых разрабатывается отдельно. Каждый модуль предоставляется клиенту по завершению работ над ним. Подобный подход позволяет сделать разработку наиболее удобной и полезной для заказчика, а также избежать излишних затрат времени. Также инкрементная модель может снизить «травматический» эффект для предприятия от внедрения готового продукта целиком.

Первыми разрабатываются модули с критически важной функциональностью, набор которых согласован с заказчиком. Когда некоторая совокупность модулей разработана, формируется первая (и последующие) версии, а вместо нереализованной функциональности в текущей версии используются так называемые «заглушки». Далее «заглушки» заменяются готовыми модулями. Подобный подход позволяет совершать поставки каждого готового модуля заказчику.

Сравним преимущества и недостатки инкрементной модели разработки ПО:

Преимущества	Недостатки
Ускоряет вывод новой версии продукта на рынок или предоставление новой версии продукта заказчику. Функциональность продукта расширяется при выполнении каждого инкремента	Не предусматриваются итерации внутри каждого инкремента. Для определения инкрементов функциональность продукта должна быть определена в начале жизненного цикла

2.3.7. Эволюционная модель развития

В изложенном ранее материале были рассмотрены модели разработки программного обеспечения. На практике, если речь идет не только о создании, но и развитии ПО, а также информационной системы в целом, такой подход часто называется эволюционной моделью развития. В работах Тома Гилба в 1976 г. был впервые использован этот термин. «Эволюция — прием, предназначенный для создания видимости стабильности. Шансы успешного создания сложной системы будут максимальными, если она реализуется в серии небольших шагов и, если каждый шаг включает в себе четко определенный успех, а также возможность отката к предыдущему успешному этапу в случае неудачи. Перед тем, как пустить в дело все ресурсы, предназначенные для создания системы, разработчик имеет возможность получать из реального мира сигналы обратной связи и исправлять возможные ошибки в проекте...» Эволюционная модель может сочетать в себе элементы каскадной и итерационной моделей на отдельных этапах, а также может применять метод прототипирования. В процессе эксплуатации первоначальной функциональности программных продуктов возникают новые и новые требования, которые реализуются как в существующем программном обеспечении, так и в создании новых программных продуктов. В дальнейшем осуществляется интеграция вновь созданных программных продуктов в общую функциональность информационной системы.

Отличительной особенностью такой модели является прохождение информационной системы через фазу эксплуатации, что будет отличать такую модель от модели разработки. Версии, которые обозначены на этой модели, являются законченными версиями, которые переданы в эксплуатацию.

Важно отметить, что в международной и российской практике все рассмотренные модели очень часто сводят к двум типам моделей: каскадной и итерационной, которую в российской практике часто называют спиральной. В разных организациях специалисты используют различный профессиональный сленг, который иногда вносит путаницу в названия этих моделей. В заключение автор хотел бы процитировать слова Мартина Фаулера, написанные им в 2004 г.: «Итеративную разработку называют по-разному: инкре-

ментальной, спиральной, эволюционной и постепенной. Разные люди вкладывают в эти термины разный смысл, но эти различия не имеют широкого признания и не так важны, как противостояние итеративного метода и метода водопада».

Контрольные вопросы и задания

1. Назовите этапы, которые включает в себя жизненный цикл информационной системы.
2. Каковы основные модели ЖЦИС?
3. Как выглядит жизненный цикл ИС в COBIT?
4. Какие области знаний рассматриваются в SWEBOOK?
5. Какие существуют стандарты жизненного цикла ИС?
6. Какие существуют основные модели ЖЦПО?
7. В чем различие между каскадной и спиральной моделями ЖЦПО?
8. В каких случаях и почему применяется каскадная модель ЖЦПО с промежуточным контролем?
9. Когда эффективнее всего может быть применена V-модель разработки ПО?

Рекомендуемая литература

1. Адизес, И. К. Управление жизненным циклом корпораций / И. К. Адизес. — Санкт-Петербург : Питер, 2008.
2. COBIT 5.0. — ISACA, 2012.
3. Guide to Software Engineering Body of Knowledge (SWEBOOK): Version 3.0. — IEEE Computer Society, 2014.